NAIC 2026 · INNOVATION & ENTERPRISE · PROBLEM STATEMENT 01: DEVELOPER INFRASTRUCTURE

N-ATLAS Dev Kit

Add Nigeria's own AI to any app in minutes

Technical documentation · version 0.1 · 12 October 2026



Team	Peachel Innovations (Peachel Innovations Limited)
Repository	github.com/Akindats1/natlas-devkit (MIT)
npm package	@akindats/natlas
Live playground	exalted-herbal-bulge.ngrok-free.dev (API key on request)
Run on Colab	server/natlas_colab.ipynb (official NCAIR1/N-ATLaS weights)

N-ATLaS is an initiative of the Federal Ministry of Communications, Innovation and Digital Economy, and powered by Awarri Technologies.

1 Overview

The N-ATLAS Dev Kit lets any developer add N-ATLAS, Nigeria's multilingual large language model (English, Hausa, Igbo, Yoruba), to an application without machine-learning expertise, Python, or a large GPU.

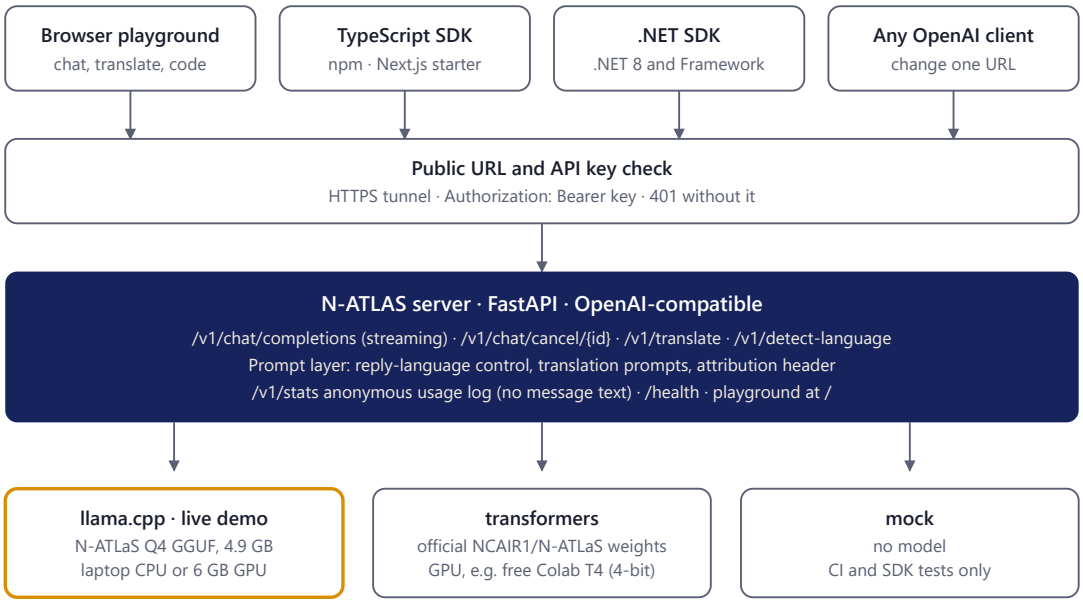
The problem

N-ATLAS is open source but hard to build with. There is no hosted endpoint, no official SDK and no starter app. A developer must download up to 16 GB of weights, write Python inference code, handle the Llama-3 chat template, and build, host and secure their own API. Most Nigerian developers work on laptops and build in JavaScript or C#.

The solution: five parts

Part	What it does	Status
OpenAI-compatible server `server/`	Runs N-ATLAS behind `/v1/chat/completions` (with streaming), `/v1/translate` and `/v1/detect-language`. A `reply_language` option forces English, Hausa, Igbo or Yoruba. Any OpenAI-based app switches by changing one URL. Runs on a laptop, in Docker, or on free Google Colab.	Live; 9 automated tests pass
TypeScript SDK `sdk-typescript/`	`@akindats/natlas`: zero dependencies; chat, streaming, translation, language detection. Node 18+, browsers, Next.js.	Published on npm (0.1.0)
.NET SDK `sdk-dotnet/`	`NAtlas`: the same features for C#; targets .NET 8 and .NET Standard 2.0, so it also runs in .NET Framework apps such as Revit and AutoCAD plugins.	In repository
Next.js starter `starter-nextjs/`	A complete multilingual chat app with streaming and a reply-language selector. Clone, set one variable, ship.	Runs; type-checks
Browser playground `/` on the server	Chat and translate in four languages; a "Build with it" panel shows copy-ready curl, TypeScript, C# and Python code for each request.	Live

2 Architecture



Every client, from the playground to existing OpenAI code, calls the same server; the server hands generation to the configured backend.

How a request flows

1. An app calls an SDK (or any OpenAI client) with messages and, optionally, a reply language.
2. The server checks the API key and validates the request. If `reply_language` is set, it injects a system prompt that forces the reply language and repeats it as a short reminder on the latest user message (after a few English turns the model otherwise follows the conversation).
3. The configured backend runs N-ATLAS: **llamacpp** (quantised GGUF), **transformers** (official weights, as in NCAIR's model card) or **mock**.
4. The reply returns in the standard OpenAI shape, or token by token as Server-Sent Events when `stream: true`. Streaming runs on a worker thread that can be cancelled by id.
5. An anonymous log line records endpoint, client name, detected input language, latency and token counts, never the message text. `/v1/stats` summarises it.

Design decisions

Decision	Why
OpenAI-compatible API	Most developers already use OpenAI-style clients; compatibility means no new code to adopt N-ATLAS.
Quantised GGUF (Q4) by default	Runs on an ordinary laptop; most Nigerian developers do not have large GPUs.
Official-weights backend too	Reference behaviour for teams with GPUs; follows the model card exactly.
Zero-dependency TypeScript SDK	Works in Node, browsers, edge runtimes and Next.js with nothing extra to install.
.NET SDK for .NET 8 + .NET Standard 2.0	Reaches enterprise, desktop and engineering software, including Revit and AutoCAD plugins on .NET Framework.
Single-model lock	An 8B model handles one generation at a time safely; scale out with more instances.

Decision	Why
Cancel by id	A stopped answer frees the model at once, even behind a proxy that never reports the client's disconnect (measured: next request answered in 1.1 s instead of waiting over 120 s).
Attribution built in	Response header, SDK constants and UI footers carry the required N-ATLaS attribution.

3 Quickstart

Start the server (pick one)

Option	Commands

A. Laptop (CPU, ~6 GB free RAM; Python 3.10–3.12)	`cd server`
	`pip install -r requirements.txt`
	`python -m natlas_server` (downloads the 4.9 GB model on first run; `NATLAS_GPU_LAYERS=-1` puts it on a GPU)

B. Docker	`cd server && docker compose up`
C. Free Colab GPU, official weights	Open `server/natlas_colab.ipynb` in Colab, choose a T4 GPU, add a Hugging Face token as the `HF_TOKEN` secret, run all cells.
D. Mock mode (no model)	`NATLAS_BACKEND=mock python -m natlas_server`

Then open `http://localhost:8000` for the playground or `/docs` for interactive API documentation. To start from the chat app instead: `cd starter-nextjs && cp .env.example .env.local && npm install && npm run dev`, with `NATLAS_BASE_URL` and `NATLAS_API_KEY` set in `.env.local`.

Call it

```
# curl: reply in Hausa
curl http://localhost:8000/v1/chat/completions \
  -H "Authorization: Bearer $NATLAS_API_KEY" -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"Explain inflation simply"}],"reply_language":"ha"}'
```

```
// TypeScript: npm install @akindats/natlas
import { NAtlas } from "@akindats/natlas";
const natlas = new NAtlas({ baseUrl: "http://localhost:8000", apiKey: process.env.NATLAS_API_KEY });
await natlas.ask("Explain inflation simply", { replyIn: "ha" });
await natlas.translate("Your order has been delivered", "ig");
for await (const t of natlas.stream("Tell me about Lagos")) process.stdout.write(t);
```

```
// C# / .NET
using NAtlas;
using var natlas = new NAtlasClient("http://localhost:8000", apiKey);
var answer = await natlas.AskAsync("Explain inflation simply", new ChatOptions { ReplyIn = Language.Hausa });
var yo = await natlas.TranslateAsync("Concrete pouring starts at 7am", Language.Yoruba);
```

```
# Existing OpenAI code (Python): change only the base URL
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key=NATLAS_API_KEY)
client.chat.completions.create(model="n-atlas", messages=[{"role": "user", "content": "Sannu"}])
```

4 API reference

All `/v1` routes require `Authorization: Bearer <key>` when the server is started with `NATLAS_API_KEY`. Every response carries `X-Powered-By: N-ATLaS (Awarri Technologies / FMCIDE)`. Clients may send `X-Client-Name` to label their requests in the usage log.

Method	Path	Purpose
POST	`/v1/chat/completions`	OpenAI-compatible chat; `stream: true` returns Server-Sent Events ending in `data: [DONE]`.
POST	`/v1/chat/cancel/{id}`	Stops a streaming answer by the `id` sent in its chunks; returns `{ id, cancelled }`.
POST	`/v1/translate`	`{ text, target, source? }` → `{ translation, source, target, usage }`
POST	`/v1/detect-language`	`{ text }` → `{ language, name, confidence, scores }` (fast heuristic for routing)
GET	`/v1/models`	Lists the model id `n-atlas`.
GET	`/v1/stats`	Anonymous usage summary: totals by endpoint, client and input language; average latency; first and last request.
GET	`/health`	Liveness and active backend, e.g. `{"status":"ok","backend":"llamacpp"}`. No key needed.
GET	`/`, `/playground`	Browser playground (browsers get HTML; API clients requesting `/` get JSON server info).
GET	`/docs`	Interactive OpenAPI documentation.

Chat request fields

Field	Default	Meaning
`messages`	required	OpenAI-style list of `{ role, content }` (system, user, assistant).
`reply_language`	none	N-ATLAS extension: `en`, `ha`, `ig` or `yo` (or the full name). Anything else returns 400.
`stream`	`false`	Stream tokens as Server-Sent Events.
`max_tokens`	`512` (1–4096)	Maximum reply length.
`temperature`, `top_p`	`0.1`, `0.95`	Sampling, as recommended in NCAIR's model card.
`stop`, `model`	none, `n-atlas`	Stop sequences; model name accepted for OpenAI compatibility.

Errors

`400` unsupported language or invalid request · `401` missing or wrong API key · `422` malformed body. The SDKs raise `NAtlasError` (TypeScript) or `NAtlasException` (.NET) with the HTTP status and a readable message, including a clear error when the server cannot be reached.

5 SDKs

TypeScript: `@akindats/natlas`

`new NAtlas({ baseUrl, apiKey, clientName })`, each falling back to `NATLAS_BASE_URL`, `NATLAS_API_KEY`, `NATLAS_CLIENT_NAME`.

- `ask(prompt, { replyIn })` → text
- `chat(messages, opts)` → text and usage
- `stream(messages | prompt, opts)` → async iterator of tokens
- `translate(text, target, source?)`
- `detectLanguage(text)`, `health()`
- `ATTRIBUTION`, `LANGUAGES`, `prompts`

.NET: `NAtlas`

`new NAtlasClient(baseUrl, apiKey, clientName)`; `baseUrl` and `apiKey` fall back to `NATLAS_BASE_URL` and `NATLAS_API_KEY`.

- `AskAsync(prompt, ChatOptions)`
- `ChatAsync(messages, ChatOptions)`
- `StreamAsync(...)` → `IAsyncEnumerable<string>`
- `TranslateAsync(text, Language)`
- `DetectLanguageAsync(text)`, `IsHealthyAsync()`
- `Language`: English, Hausa, Igbo, Yoruba

6 Configuration

Variable	Default	Meaning
`NATLAS_BACKEND`	`llamacpp`	`llamacpp`, `transformers` or `mock`
`NATLAS_MODEL_PATH`	none	Use a local `.gguf` file instead of downloading
`NATLAS_GGUF_REPO` / `_FILE`	`tosinamuda/N-ATLaS-GGUF`, Q4_K_M	Quantised build to download (Q8 for higher quality)
`NATLAS_GPU_LAYERS`	`0`	`-1` puts every layer on the GPU
`NATLAS_HF_MODEL`	`NCAIR1/N-ATLaS`	Official weights for the `transformers` backend
`NATLAS_LOAD_4BIT`	off	`1` loads the official weights in 4-bit (fits a free T4)
`NATLAS_API_KEY`	none	When set, clients must send `Authorization: Bearer <key>`
`NATLAS_LOG`	`usage.jsonl`	Anonymous usage log path (no message text)

7 N-ATLAS integration

N-ATLAS is the only model the kit uses; nothing wraps another general-purpose model.

Backend	Model loaded	Code
`llamacpp`	Q4_K_M GGUF quantisation of `NCAIR1/N-ATLaS` (4.9 GB; SHA-256 verified against Hugging Face)	`backends.py` → `LlamaCppBackend`
`transformers`	Official `NCAIR1/N-ATLaS` weights, optional 4-bit	`backends.py` → `TransformersBackend`

Faithful to the model card: Llama-3 chat template with `date_string`, `repetition_penalty` 1.12, low temperature, and the Awarri system prompt. Language features are limited to the four languages N-ATLAS supports.

```
GPU:          Tesla T4, 6031 MiB, 15360 MiB
Model:        NCAIR1/N-ATLaS (official weights, 4-bit)
Health:       {'status': 'ok', 'backend': 'transformers'}
Powered by:   N-ATLaS (Awarri Technologies / FMCIDE)
Usage:        {'prompt_tokens': 73, 'completion_tokens': 200, 'total_tokens': 273} in 19.2s

Q (Hausa): menene ake nufi da gwagwarmaya
A: Gwagwarmaya na iya nufin wasu abubuwa daban-daban bisa ga mahallin:

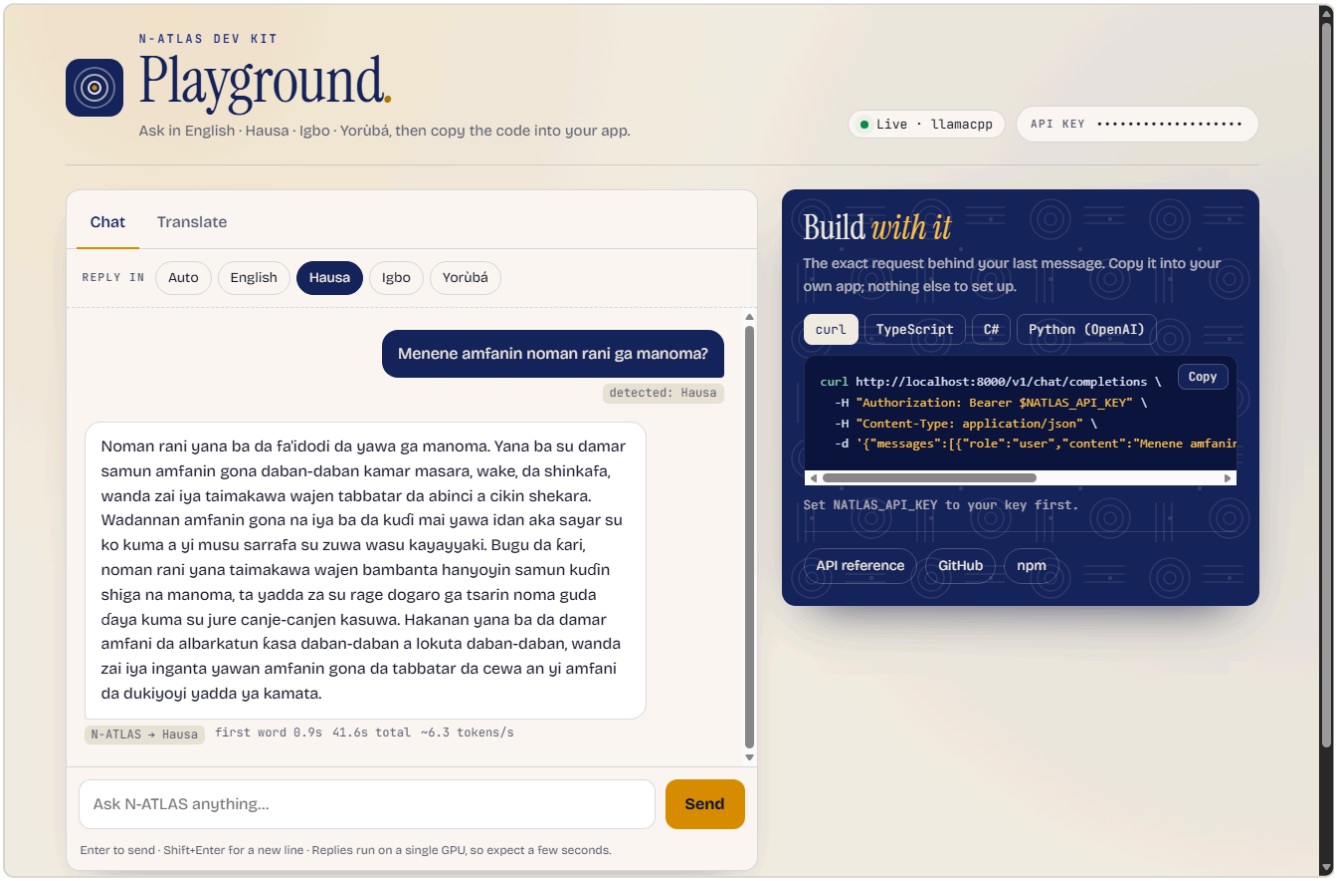
1. **Matakan Juriya**: Zai iya zama matakin juriya ko kuma kokarin shawo kan matsaloli, kalubale, ko cikas.

2. **Tashin Hankali**: Hakanan ana amfani da shi ne don bayyana tashin hankali ko rikici, ko dai na sirri (misali, gwagwarmayar ciki) ko na al'umma (misali, gwagwarmayar siyasa).

3. **Yaki ko Soja**: A cikin mahallin soja ko na yaki, gwagwarmaya na nufin fada ko kuma babban taron yaki.

4. **Wasanni**: A cikin wasannin motsa jiki
```

Evidence 1: official weights on a free Colab T4. `NCAIR1/N-ATLaS` loaded in 4-bit by the `transformers` backend (6 GB of 15 GB GPU memory), attribution header present, and a Hausa answer to the model card's own question: 200 tokens in 19.2 s.



Evidence 2: quantised build on a laptop GPU. The playground answering a Hausa question through the `llamacpp` backend on a 6 GB GTX 1660 Ti: first word in 0.9 s, about 6 tokens/s.

8 Security, privacy and licence

- **Access control:** an API key protects any public deployment; requests without it get HTTP 401. The key lives in a git-ignored `.env` file.
- **Privacy:** the usage log stores endpoint, client name, detected language, latency and token counts only. No message text is stored.
- **Secrets:** model weights, keys, tokens and `.env` files are excluded from the repository by `.gitignore`.
- **Licence:** the kit's code is MIT-licensed. The model stays under the N-ATLaS Terms of Use: free for research, education, civic and community use, up to 1,000 active end-users without a commercial licence, attribution required.
- **Acceptable use passed on:** the README lists the permitted uses (research, accessibility, language preservation, civic tech, education) and the prohibited ones (surveillance, discriminatory profiling, disinformation and fraud, military use, harmful or unlawful applications). The contribution guide requires contributors to respect them.

9 Testing and verification

Automated tests run against the mock backend; the full chain was also verified on the real model.

Check	Result
Server API tests (pytest, 9 tests: health, chat shape, streaming, reply language, translation, stats, playground, disconnect and cancel on a real server)	**9 of 9 pass**
TypeScript SDK tests through the public URL on the real model	**6 of 6 pass**
Fresh `npm install @akindats/natlas` in an empty folder	Health check, Igbo answer and Hausa translation returned
Playground in a headless browser	Hausa answer streamed; first word in 0.9 s; code panel updated
Reply language after English turns (Yoruba)	Answers in Yoruba (fixed by the latest-message reminder)
Stop through the public URL	Next request answered in 1.1 s (was over 120 s)
Model file integrity	SHA-256 matches the Hugging Face record
API key	Requests without the key refused (HTTP 401)

Run them: `cd server && pytest` · `cd sdk-typescript && npm test` (with a running server) · `dotnet run --project sdk-dotnet/samples/NAtlas.Demo`.

10 Known limitations and roadmap

Limitations

- Yoruba quality is lower than English, Hausa and Igbo (2.69/5 in NCAIR's own human evaluation).
- Context is about 8,000 tokens.
- Laptop speed: 2.6 tokens/s on CPU, about 6 on a 6 GB GPU; real-time apps need a larger GPU.
- `detect-language` is a heuristic for routing, not a classifier.
- One instance serves one request at a time; scale out with more instances.

Roadmap

- Speech: N-ATLAS ASR endpoints for voice-first apps.
- A Python SDK and a NuGet release of the .NET SDK.
- A hosted endpoint, so developers need no setup.
- LangChain and Vercel AI SDK adapters.
- An evaluation dashboard for per-language quality.

11 Repository layout

```
natlas-devkit/
├── server/                  OpenAI-compatible server (Python, FastAPI)
│   ├── natlas_server/       app.py (routes) · backends.py (llamacpp, transformers, mock)
│   │                        prompts.py (languages, reply control) · static/playground.html
│   ├── tests/               pytest suite
│   ├── natlas_colab.ipynb   free Colab GPU, official weights
│   └── Dockerfile, docker-compose.yml, requirements*.txt
├── sdk-typescript/          @akindats/natlas (src, tests, examples)
├── sdk-dotnet/              NAtlas (.NET 8 + .NET Standard 2.0) and a demo
├── starter-nextjs/          multilingual chat starter app
├── docs/                    ARCHITECTURE.md · NATLAS_INTEGRATION.md · VALIDATION.md
├── submission/              tester guide, screenshots, pitch deck
└── README.md · CONTRIBUTING.md · LICENSE (MIT)
```